

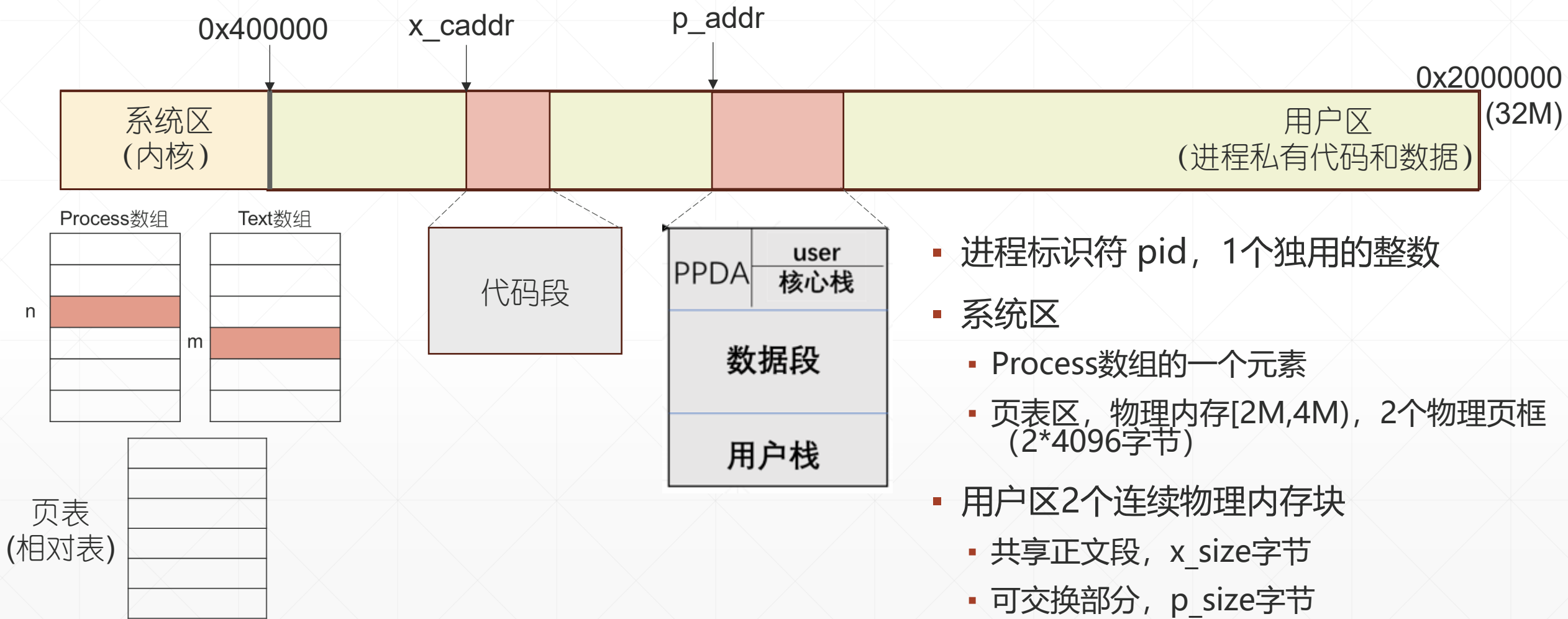
操作系统

第二章 进程管理

Unix V6+ + fork系统调用 & 系统架构

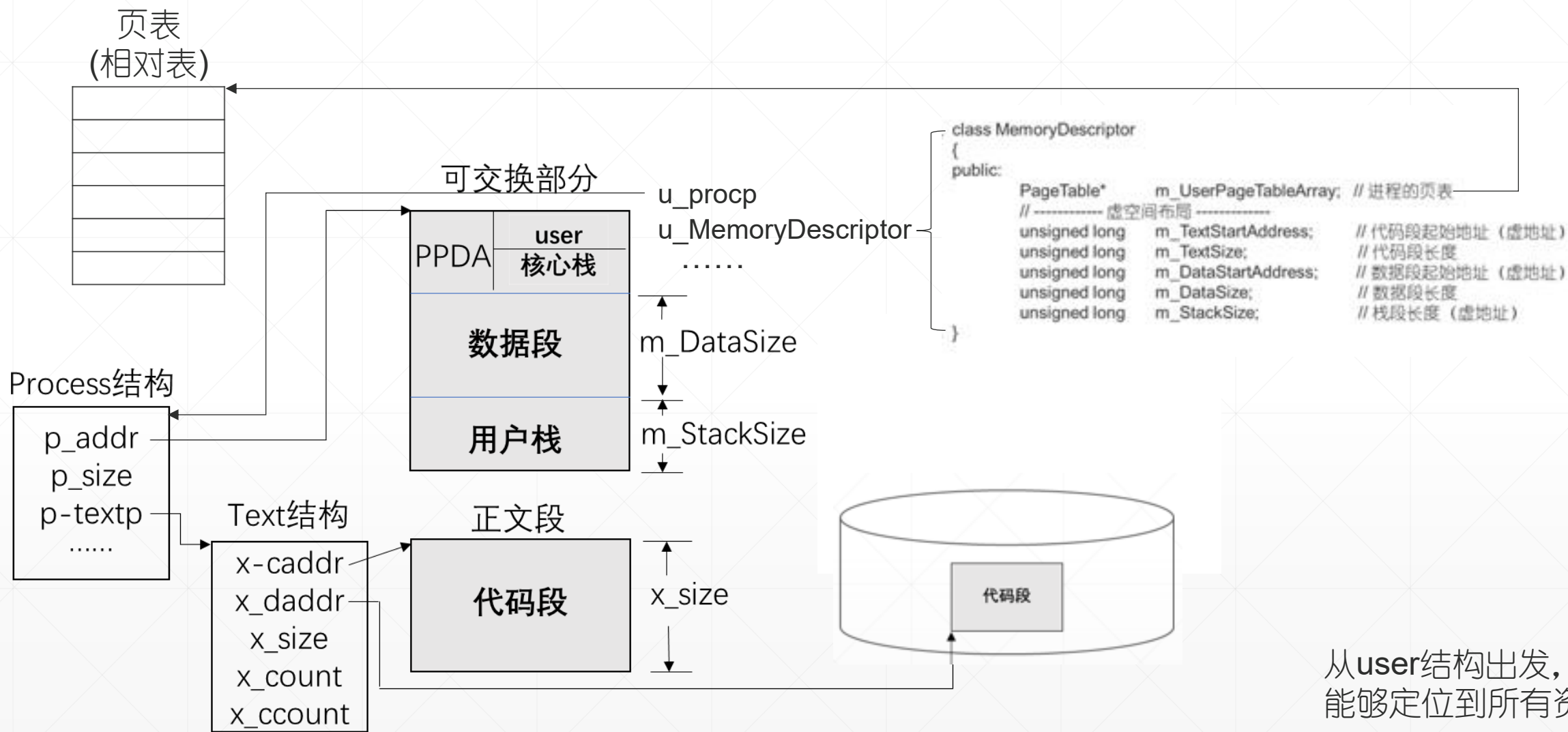
同济大学计算机系

一、进程图像使用的系统资源

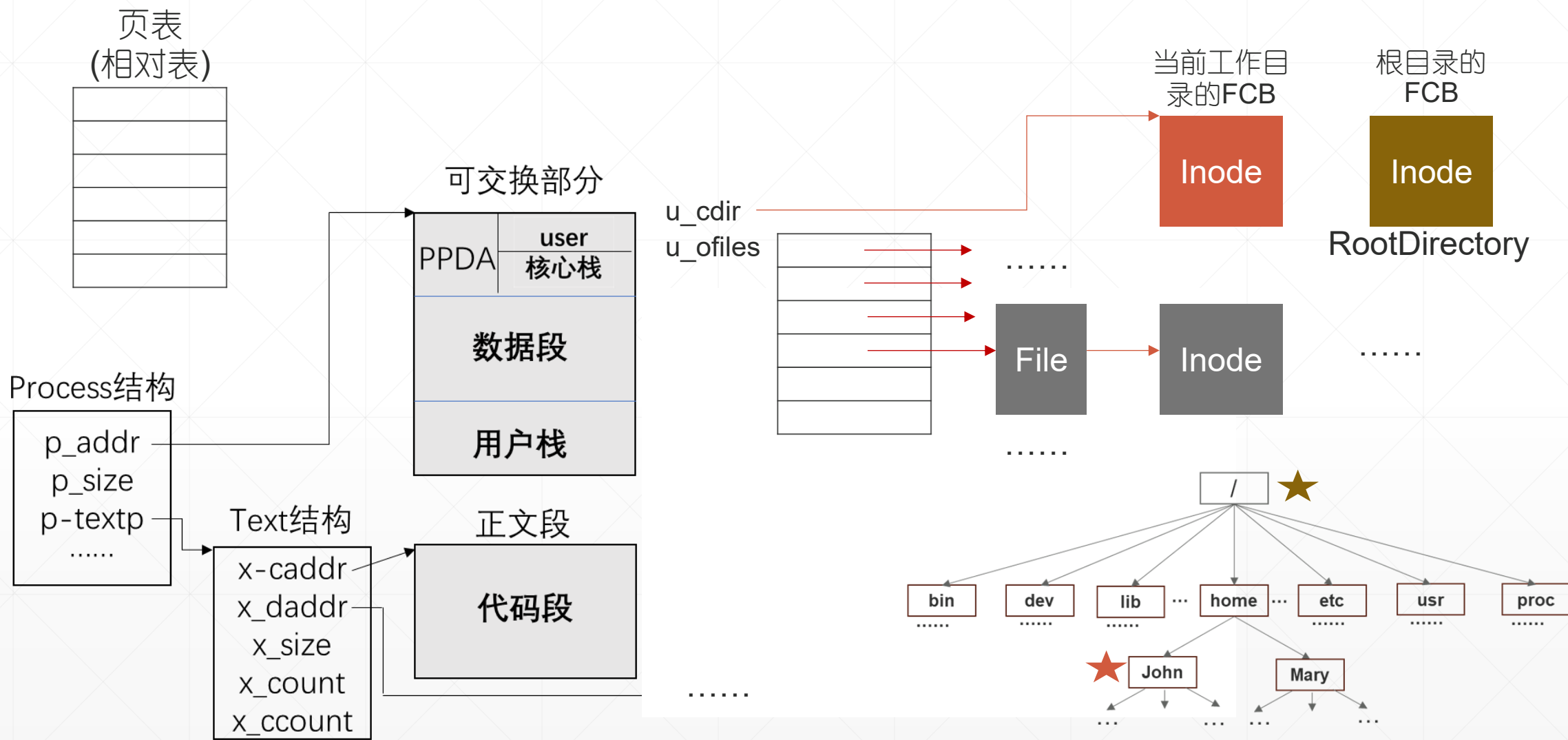


- 进程标识符 pid, 1个独用的整数
- 系统区
 - Process数组的一个元素
 - 页表区, 物理内存[2M,4M), 2个物理页框 (2*4096字节)
- 用户区2个连续物理内存块
 - 共享正文段, x_size字节
 - 可交换部分, p_size字节

从Process出发，定位分配给进程的全部资源 (1)



从Process出发，定位进程使用的资源（文件系统）





二、进程管理，核心数据结构

```
class ProcessManager // src/include/ProcessManager.h
{
    public:
        Process process[NPROC]; // 100个元素
        Text text[NTEXT]; // 50个元素
        .....
    private:
        static unsigned int m_NextUniquePid; // 为下一个新进程分配的pid号
}
```

process数组100个元素，系统最多并发100个进程。
text数组50个元素，系统最多同时执行50个应用程序。
系统初始化时， m_NextUniquePid 值为0。



process数组

p_stat != 0(SNULL)
p_stat != 0(SNULL)
p_stat != 0(SNULL)
p_stat == 0(SNULL)
.....
.....
.....
.....
.....
.....
.....
.....

Class Process

{

public:

// 1、进程标识

short p_uid; // 用户标识
int p_pid; // 进程标识
int p_ppid; // 父进程标识

// 2、分配给正文段和可交换部分的物理内存

unsigned long p_addr; // 可交换部分起始地址
unsigned int p_size; // 可交换部分的长度 (以字节单位)
Text* p_textp; // 正文段的控制块

// 3、进程状态

ProcessState p_stat; // CPU调度状态
int p_flag; // 其它状态标识。SLOAD为1表示可交换部分
SSYS为1表示系统进程 (0#进程的SSYS)

// 4、进程优先数

int p_pri; // 进程优先数
int p_nice; // 用来计算进程优先数，这是基准量
int p_cpu; // cpu使用时长，用来计算进程优先数，实现时间片轮转调度

// 5、驻留时间，是一个计时器

int p_time; // 进程在盘交换区或内存的驻留时间

// 6、进程睡眠原因

unsigned long p_wchan; // 是一个内核变量的首地址

// 7、进程收到的信号

int p_sig;
unsigned long p_sigmap;

// 8、进程的终端

TTY* p_ttyp; // 键盘输入、屏幕输出

};

enum ProcessState /* 进程状态 */

{

SNULL = 0, /* 未初始化空状态 */
SSLEEP = 1, /* 高优先权睡眠 */
SWAIT = 2, /* 低优先权睡眠 */
SRUN = 3, /* 运行、就绪状态 */
SIDL = 4, /* 进程创建时的中间状态 */
SZOMB = 5, /* 进程终止时的中间状态 */
SSTOP = 6, /* 进程正被跟踪 */

};

系统初始化完成后并发3个进程，0#进程，1#进程（init进程），2#进程（shell进程）

text数组

init程序的Text
shell程序的Text
x_iptr == NULL
x_iptr == NULL
.....
x_iptr == NULL

```

Class Text
{
public:
    int          x_daddr; // 共享正文段在盘交换区上的起始扇区号
    unsigned long x_caddr; // 共享正文段的物理内存起始地址，单位字节
    unsigned int  x_size;  // 共享正文段长度，单位字节
    Inode*        x_iptr;  // 指向可执行文件的内存inode FCB
    unsigned short x_count; // 共享正文段的进程数
    unsigned short x_ccount; // 共享该正文段且图像在内存的进程数
};
    
```

Text结构管理应用程序代码段，有多少个程序，就有多少个Text结构。
系统初始化完成后，有2个正在执行的程序，init程序和 shell程序



例1、父进程为子进程分配Process结构和p_pid

Sys_Fork()→Fork()→NewProc()

源代码: src/proc/ProcessManager.cpp

```
int ProcessManager::NewProc()
{
    Process* child = 0;
    for (int i = 0; i < ProcessManager::NPROC; i++ )
        if ( process[i].p_stat == Process::SNULL )
        {
            child = &process[i]; // 分配给子进程的process结构
            break;
        }
    if ( !child )
        Utility::Panic("No Proc Entry!"); // 内核报错

    User& u = Kernel::Instance().GetUser(); // 父进程的user结构
    Process* current = (Process*)u.u_procp; // 父进程的Process结构
    current->Clone(*child); // 父进程调用Clone函数，为子进程分配pid
    .....
}
```

```
void Process::Clone(Process& proc)
{
    .....
    proc.p_pid = ProcessManager::NextUniquePid();
    proc.p_ppid = this->p_pid; // this是父进程
    .....
}
```

```
unsigned int ProcessManager::NextUniquePid()
{
    return ProcessManager::m_NextUniquePid++;
}
```


三、Fork()创建子进程

为子进程创建一个可以执行的进程图像。
Unix的方法是克隆父进程图像

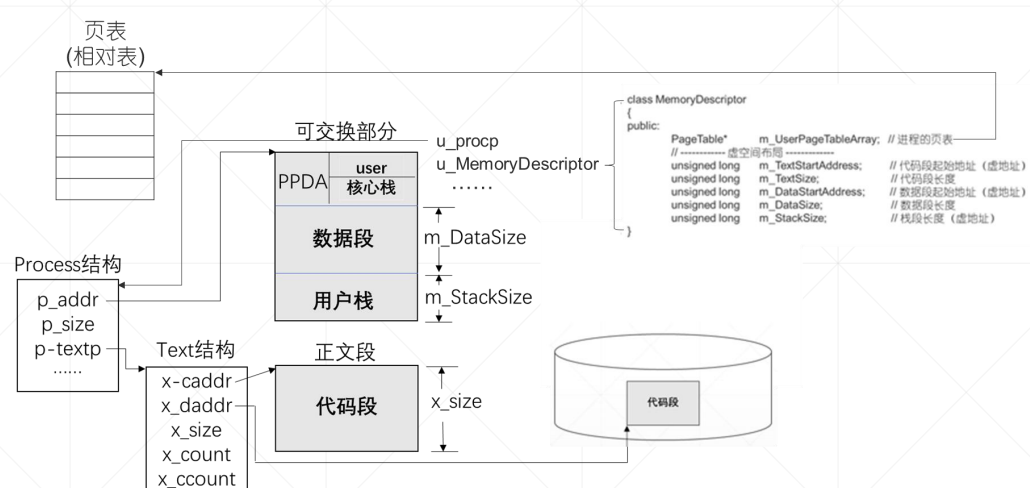


图1、执行Fork之前的父进程图像

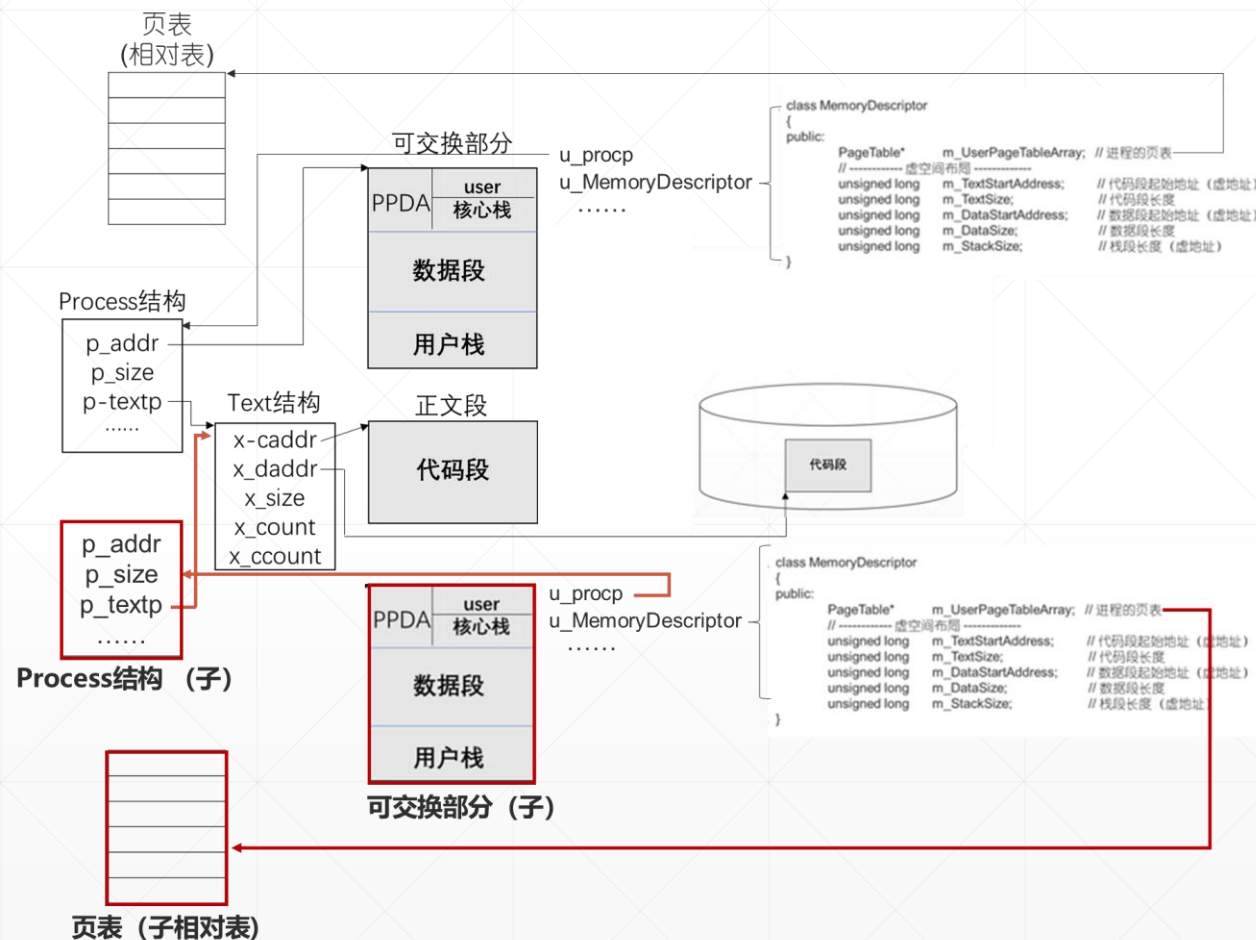
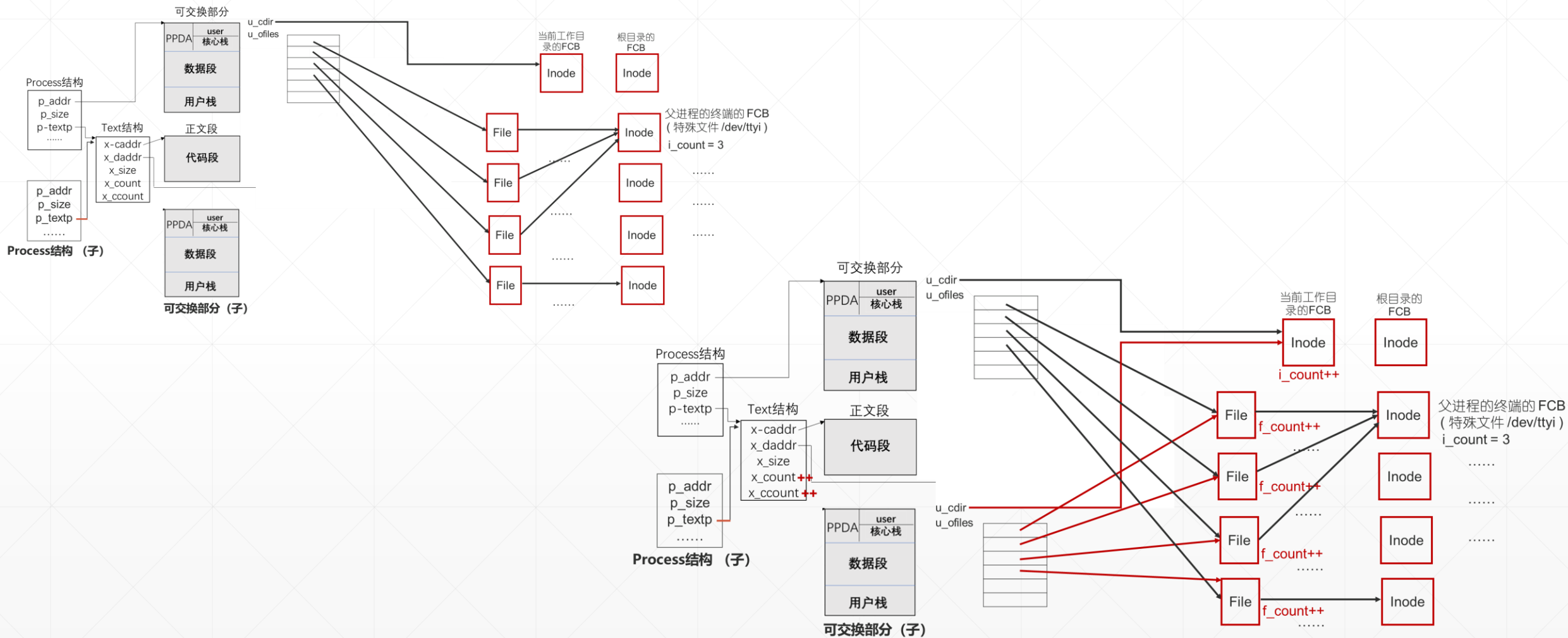


图1、父进程图像 和 子进程图像

子进程可以直接使用父进程的资源，资源计数器值加1





Fork()源码 (部分)

- 为子进程图像分配资源
- 初始化子进程图像 (Unix, 复制父进程图像)
- 修改子进程PPDA区, 令其首次执行时, 从fork()返回 0 (有技巧, 等我们讲完中断和进程切换)。



第一步：为子进程分配 Process结构 和 p_pid，累加资源计数器

接 PPT8

```
void Process::Clone(Process& proc) // proc→子进程; this→父进程
{
    User& u = Kernel::Instance().GetUser(); // 获得父进程的user结构

    proc.p_size = this->p_size; // 复制父进程的Process
    proc.p_stat = Process::SRUN;
    proc.p_flag = Process::SLOAD;
    proc.p_uid = this->p_uid;
    proc.p_ttyp = this->p_ttyp;
    proc.p_nice = this->p_nice;
    proc.p_textp = this->p_textp;

    if ( proc.p_textp != 0 )
    {
        proc.p_textp->x_count++;
        proc.p_textp->x_ccount++;
    }

    proc.p_pid = ProcessManager::NextUniquePid(); // 为子进程分配pid
    proc.p_ppid = this->p_pid; // 记住父进程
```

// 红色部分，共享父进程的正文段



```
proc.p_pri = 0;      // 调度子系统需要的参数，缓  
proc.p_time = 0;  
  
for ( int i = 0; i < OpenFiles::NOFILES; i++ )    // 遍历父进程打开文件表，File的计数器++  
{  
    File* pFile;  
    if ( (pFile = u.u_ofiles.GetF(i)) != NULL )  
    {  
        pFile->f_count++; // File的计数器++  
    }  
}  
  
u.u_error = User::NOERROR; // 是细节，理解时可以忽略  
  
u.u_cdir->i_count++; // 当前工作目录，FCB的计数器++  
}
```



第二步：为子进程分配页表，复制父进程的页表

SaveU(u.u_rsav); // for 进程切换，缓

PageTable* pgTable = u.u_MemoryDescriptor.m_UserPageTableArray; // 这是父进程的页表，记住首地址
u.u_MemoryDescriptor.Initialize(); // 为子进程分配页表

if (NULL != pgTable) // 复制页表

父进程页表

子进程页表

Utility::MemCopy(((unsigned long)pgTable, (unsigned long)u.u_MemoryDescriptor.m_UserPageTableArray,
sizeof(PageTable) * MemoryDescriptor::USER_SPACE_PAGE_TABLE_CNT);

2*4096字节



第三步：为子进程分配可交换部分，复制父进程的 ~

```
u.u_procp = child;
```

```
// userPageManager管理物理内存的用户区 [2M, 4M), 主营内存分配回收
```

```
UserPageManager& userPageManager = Kernel::Instance().GetUserPageManager();
```

```
unsigned long srcAddress = current->p_addr; // 父进程可交换部分首地址，物理地址
```

```
// 用户区，为子进程可交换部分，分配尺寸为p_size字节的连续内存块。desAddress是首地址，物理地址。
```

```
unsigned long desAddress = userPageManager.AllocMemory(current->p_size);
```

```
if ( desAddress == 0 ) // 内存不够
```

```
{
```

父进程可交换部分，复制一份在盘交换区。这是子进程的可交换部分，子进程的p_addr是盘交换区的起始扇区号。这个子进程是盘交换区上的进程，p_flag SLOAD位清0。

```
}
```



```
else // 内存分配成功
```

```
{  
    int n = current->p_size;  
    child->p_addr = desAddress; // 设置子进程的p_addr  
    while (n--)  
    {  
        Utility::CopySeg(srcAddress++, desAddress++); // 复制父进程可交换部分, 每次一个字节  
    } // CopySeg( ), 物理空间复制数据  
}
```

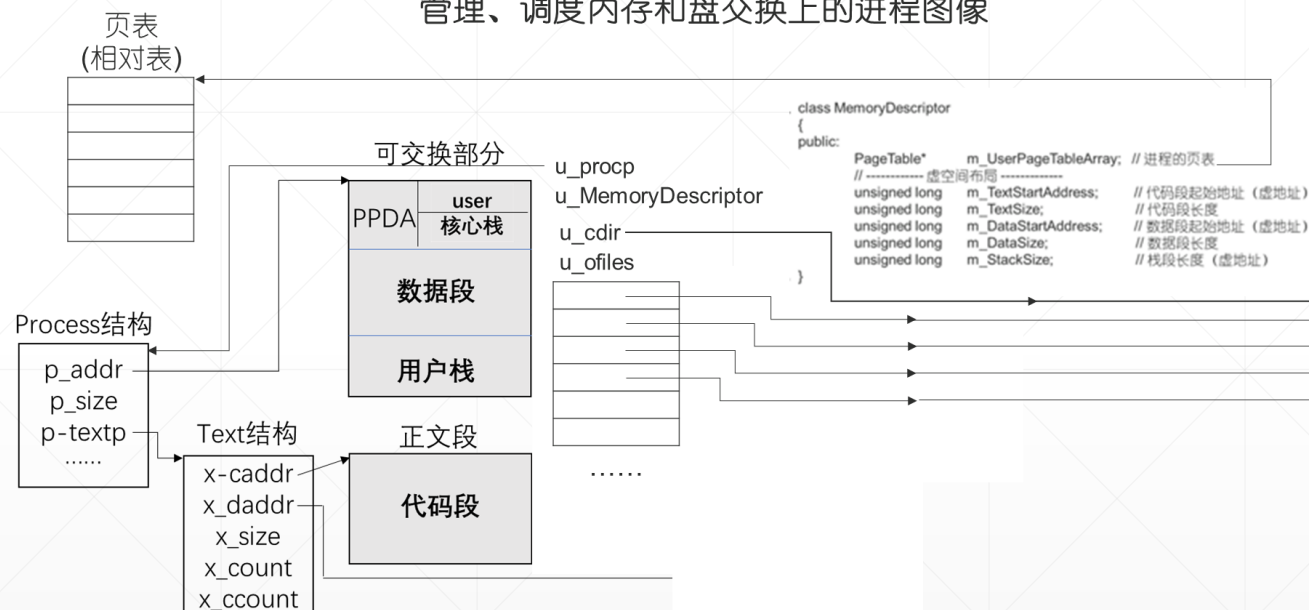
至此, 子进程图像创建完毕。只要给它CPU, 它能够运行, fork返回0。细节 缓, 等中断, 进程切换

```
// 调整父进程可交换部分 和 页表指针。父子进程完全独立。  
u.u_procp = current;  
u.u_MemoryDescriptor.m_UserPageTableArray = pgTable;  
return 0;  
}
```

Unix V6++ 系统架构

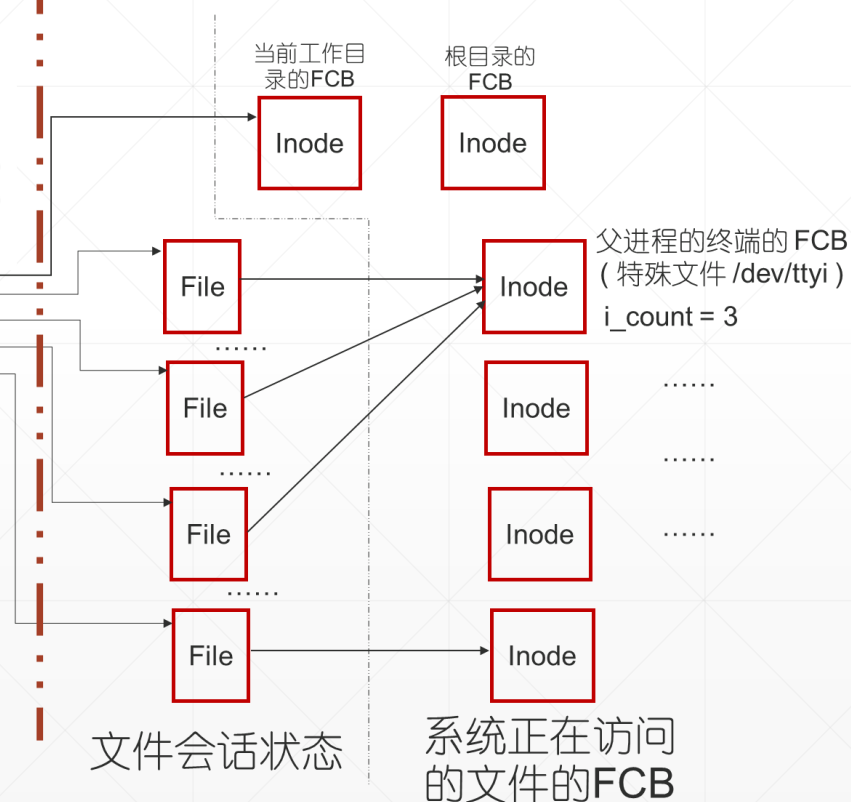
进程管理子系统 & 内存管理系统

管理、调度内存和盘交换上的进程图像



文件管理子系统

为进程提供数据服务 (数据读取 & 持久化)



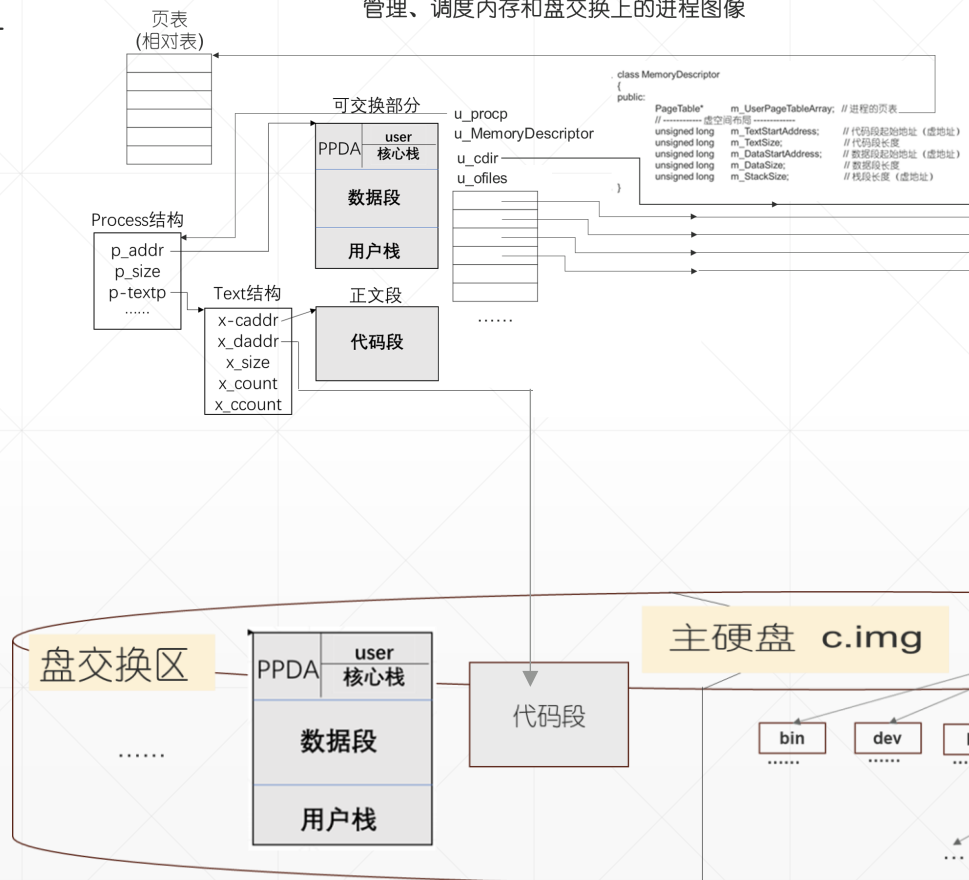
Unix V6++ 系统架构 (with 主硬盘)

核心数据结构:

- process数组
- text数组
- user结构

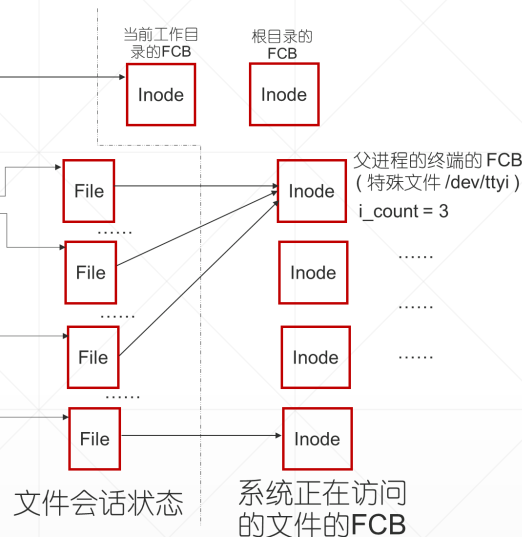
进程管理子系统 & 内存管理系统

管理、调度内存和盘交换上的进程图像



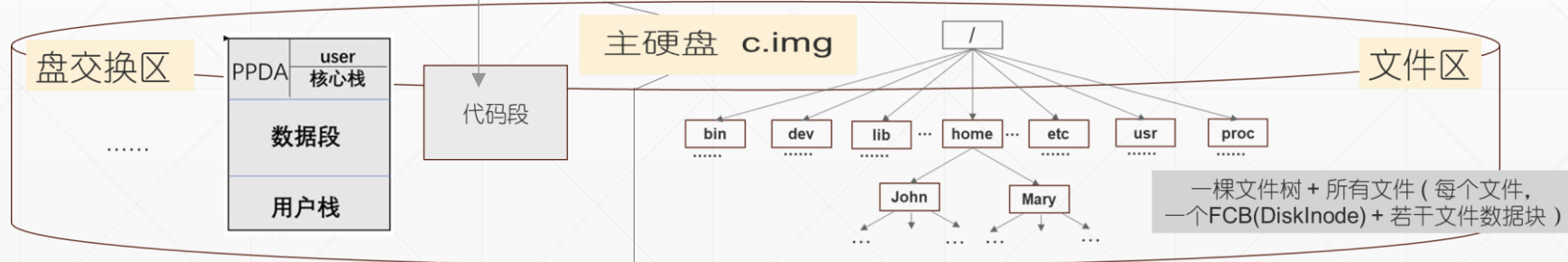
文件管理子系统

为进程提供数据服务 (数据读取 & 持久化)

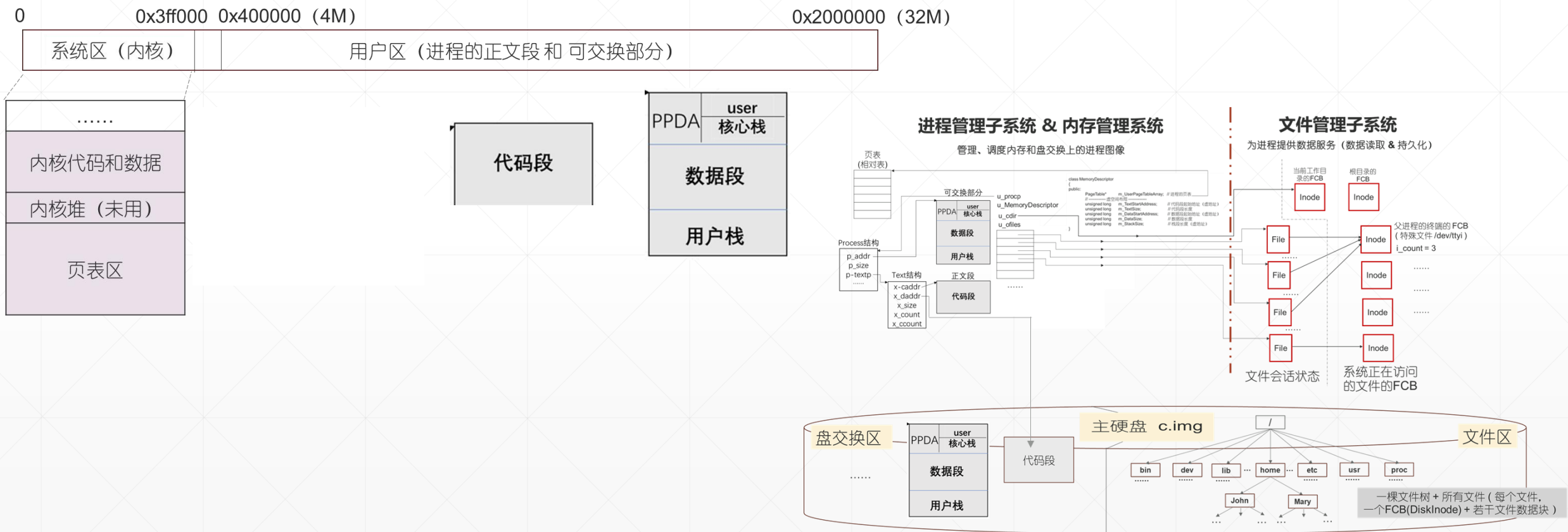


核心数据结构:

- Inode数组
- File数组
- 目录项 DirectoryEntry

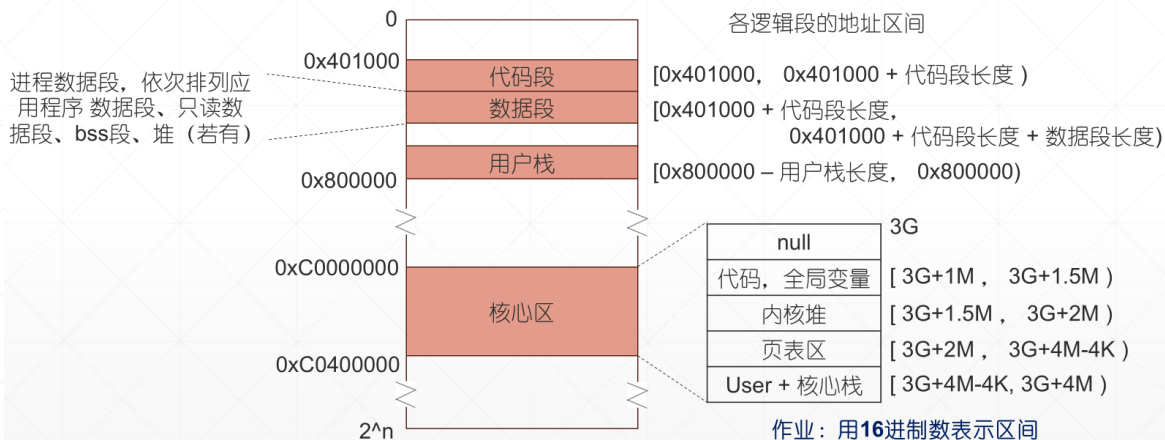


数据结构长哪儿啦？？？（物理空间）

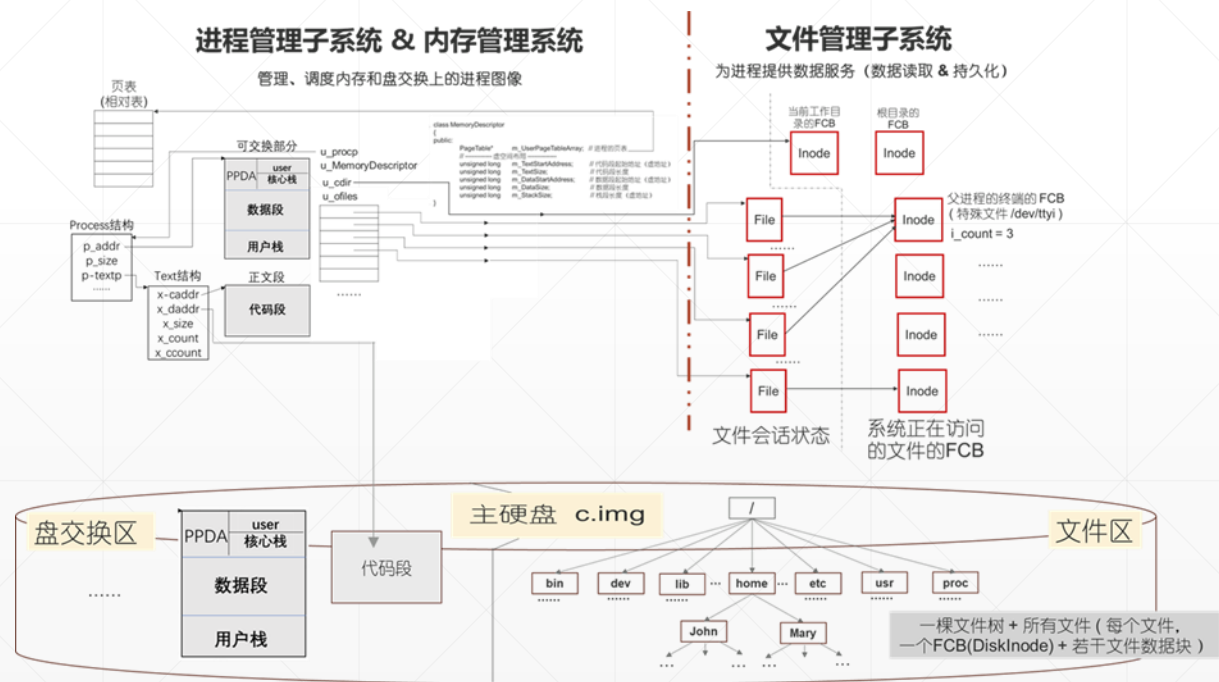


数据结构长哪儿啦？（虚空间，它们的地址）

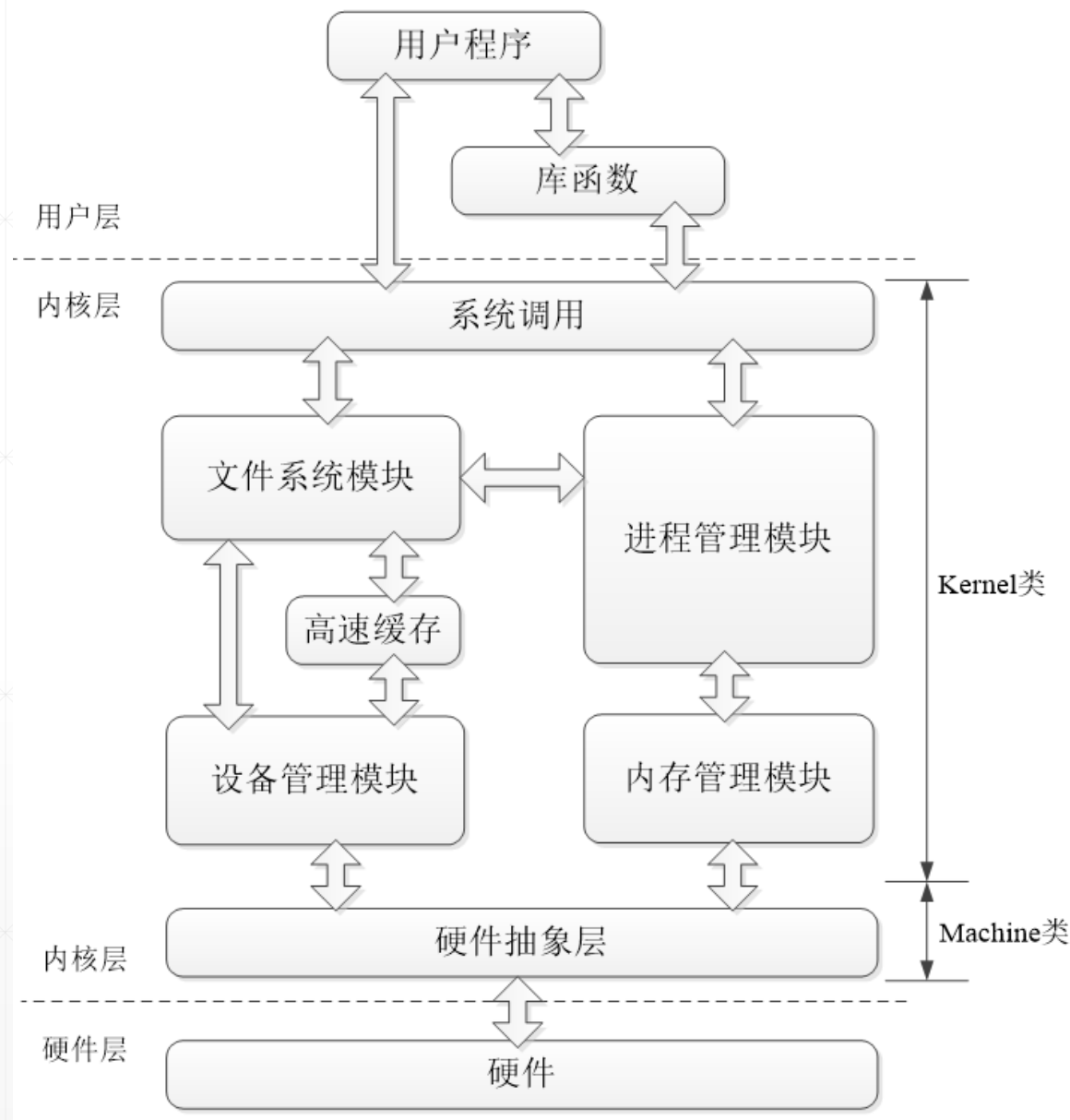
Unix V6++ 进程的虚空间



Process结构，虚地址大概是多少？
User结构，虚地址是多少？

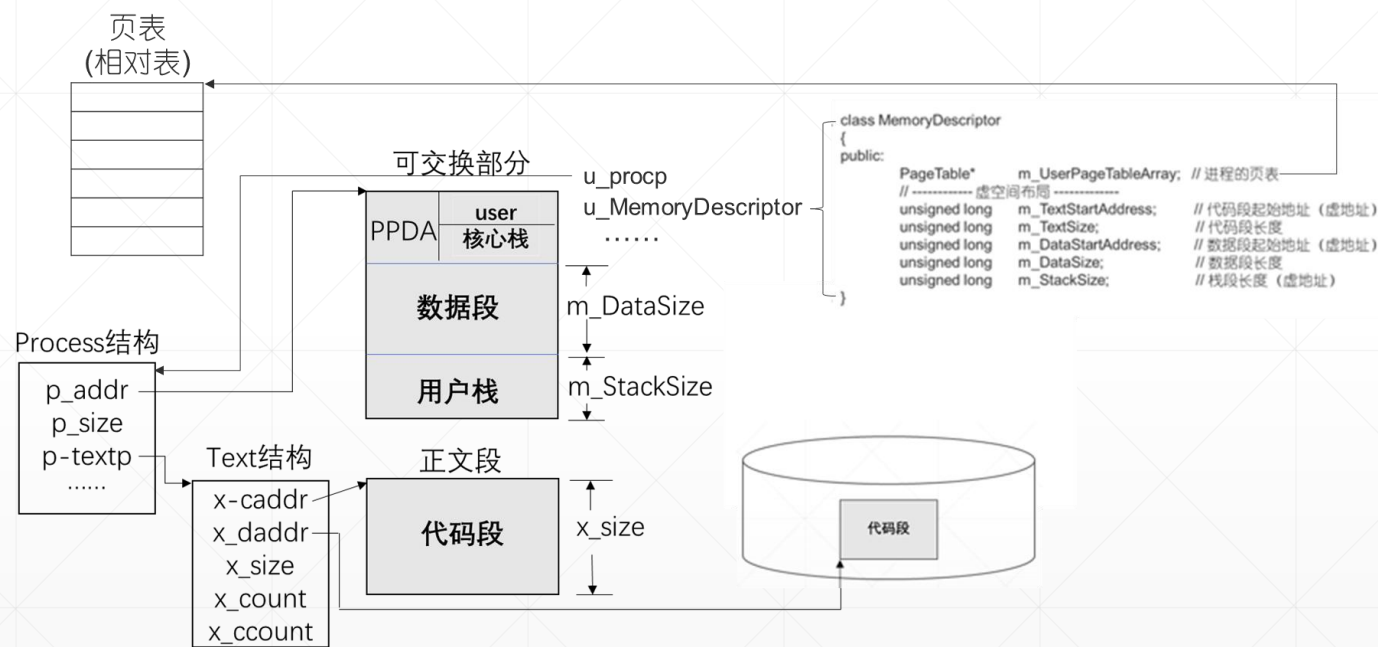


Unix V6++内核结构

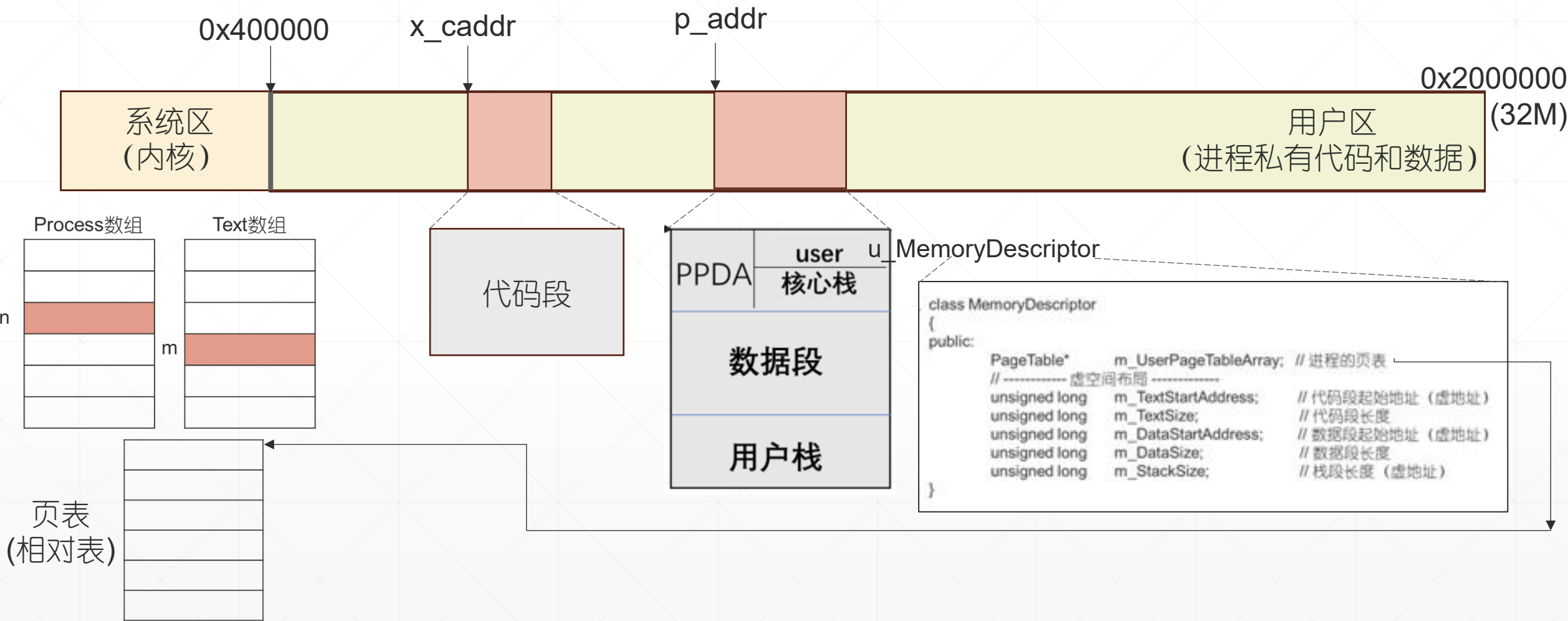


Fork()创建子进程 (一、资源分配 & 进程图像复制)

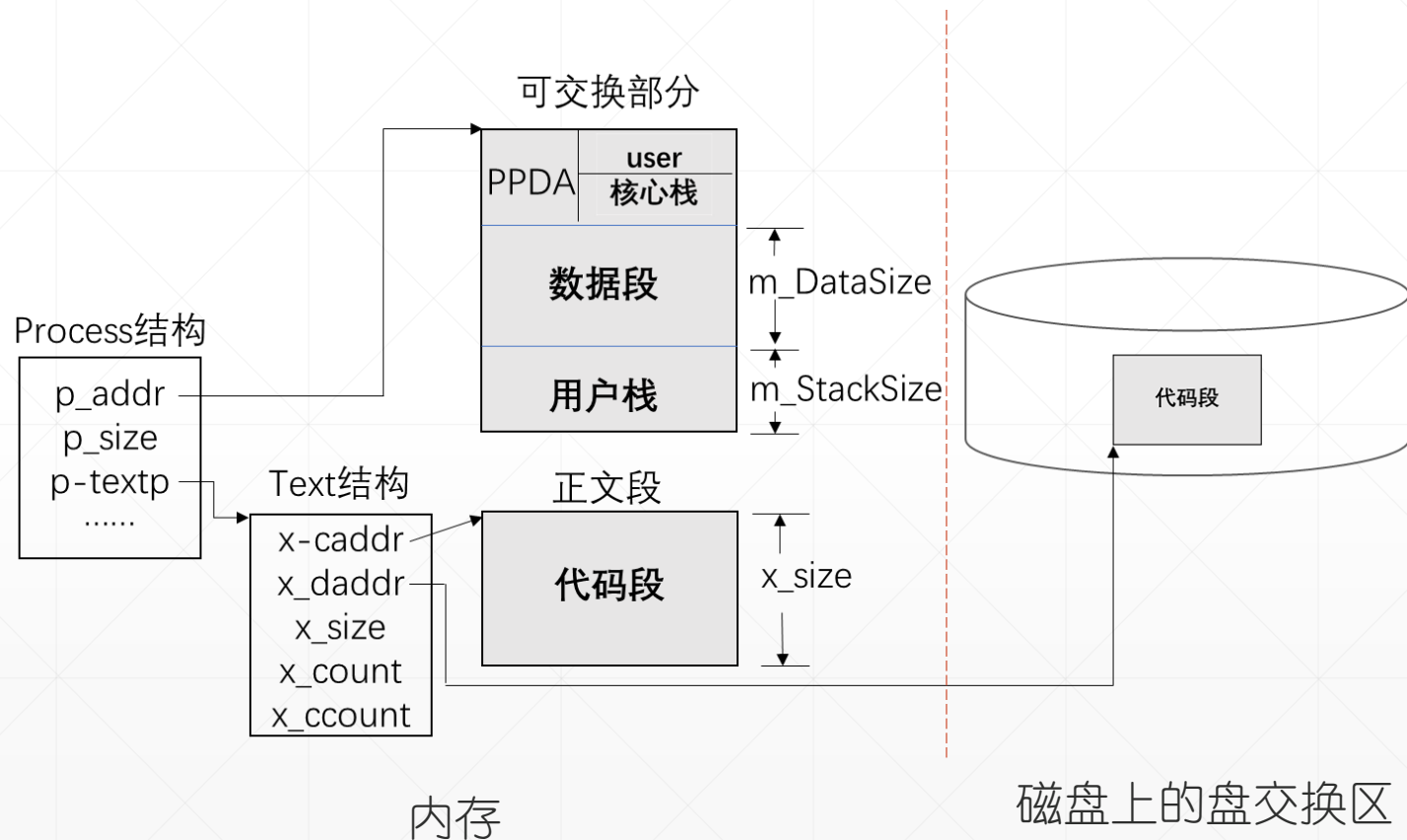
- 进程标识符 pid, 1个独用的整数
- 系统区
 - Process数组分配1个空元素
 - 页表区, 物理内存[2M,4M), 分配2张页表
- 用户区2个连续物理内存块
 - 共享正文段, x_size字节
 - 可交换部分, p_size字节



进程图像使用的系统资源



进程使用的系统资源

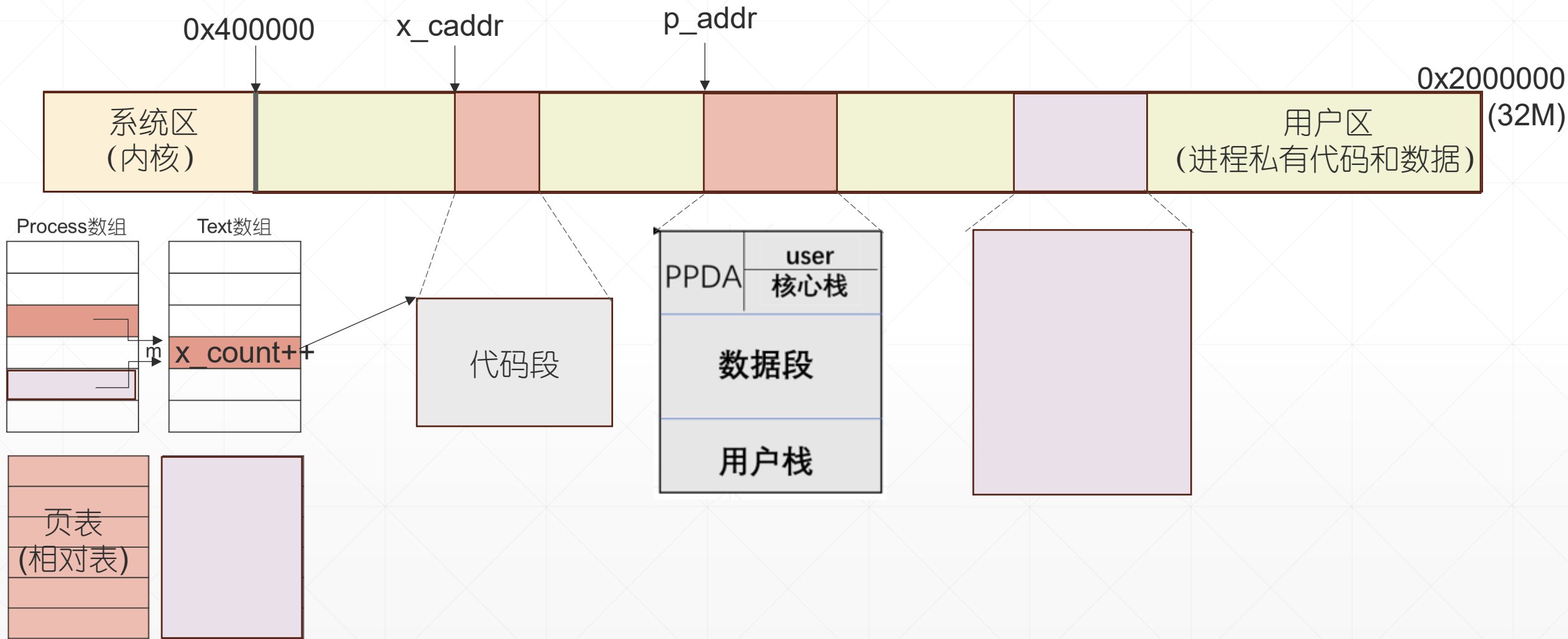


- 进程标识符 `pid`, 一个独用的整数
- 系统区
 - `Process`数组的一个元素
 - `Text`数组的一个元素
 - 2张页表
- 用户区2个连续物理内存块
 - 共享正文段, `x_size`字节
 - 可交换部分, `p_size`字节

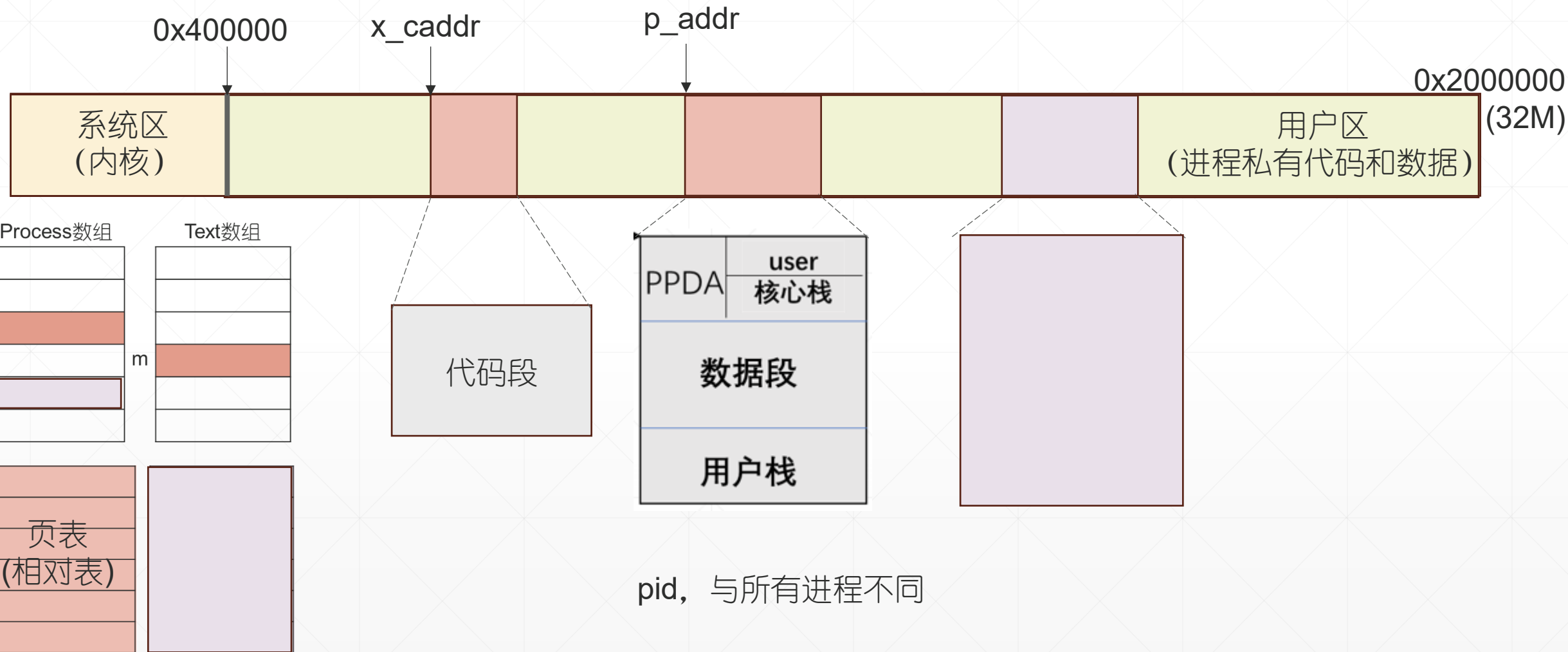


`child = fork()`, 创建子进程、复制父进程图像

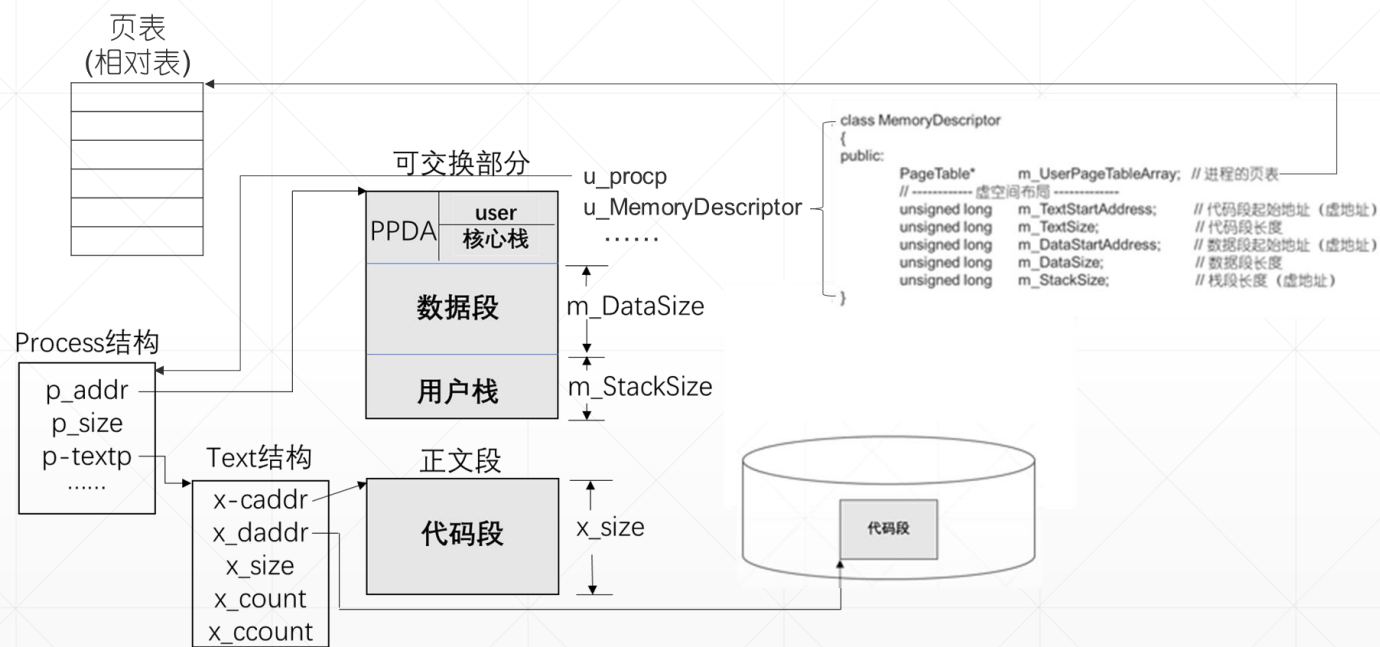
进程图像使用的系统资源

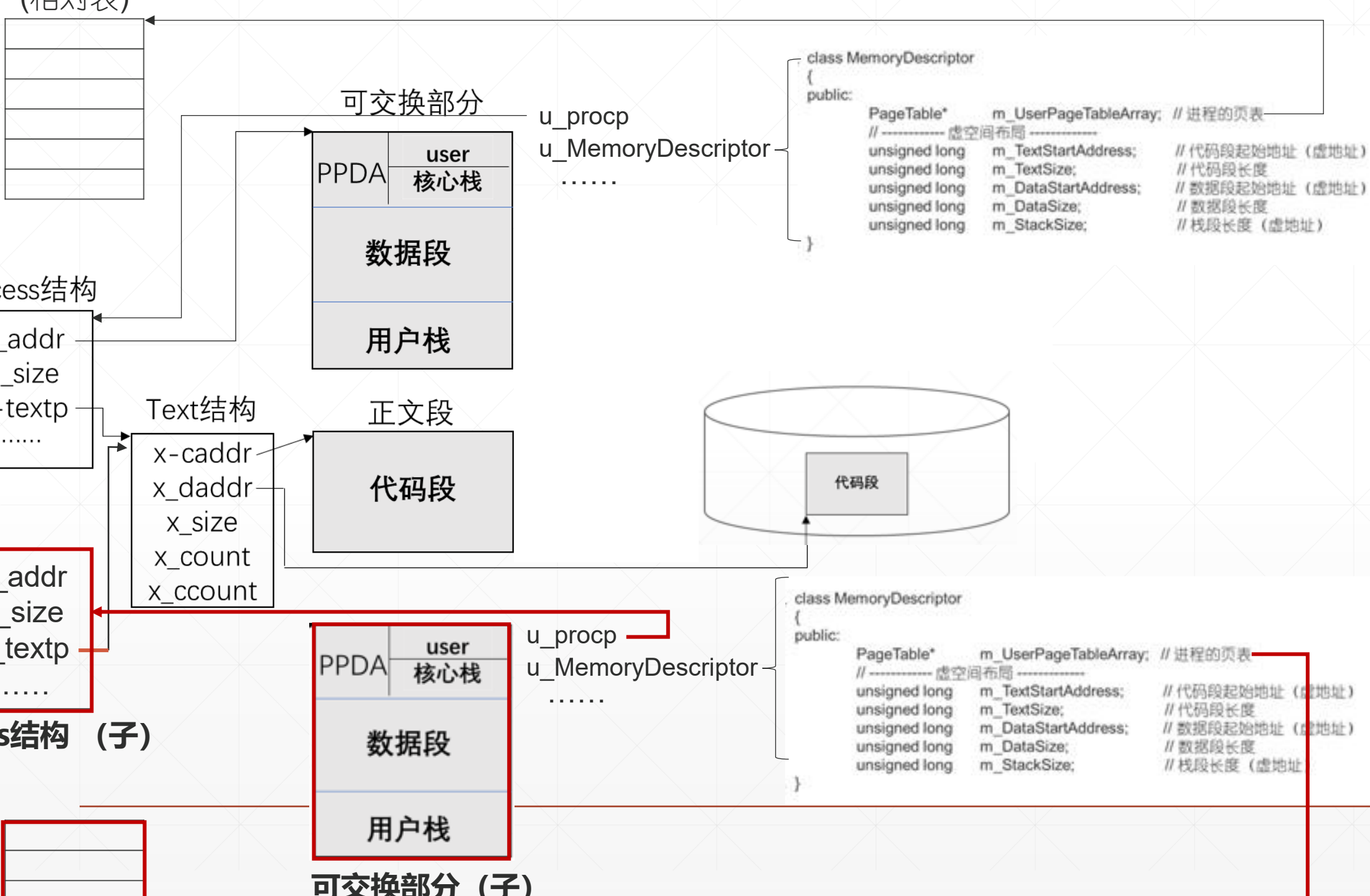


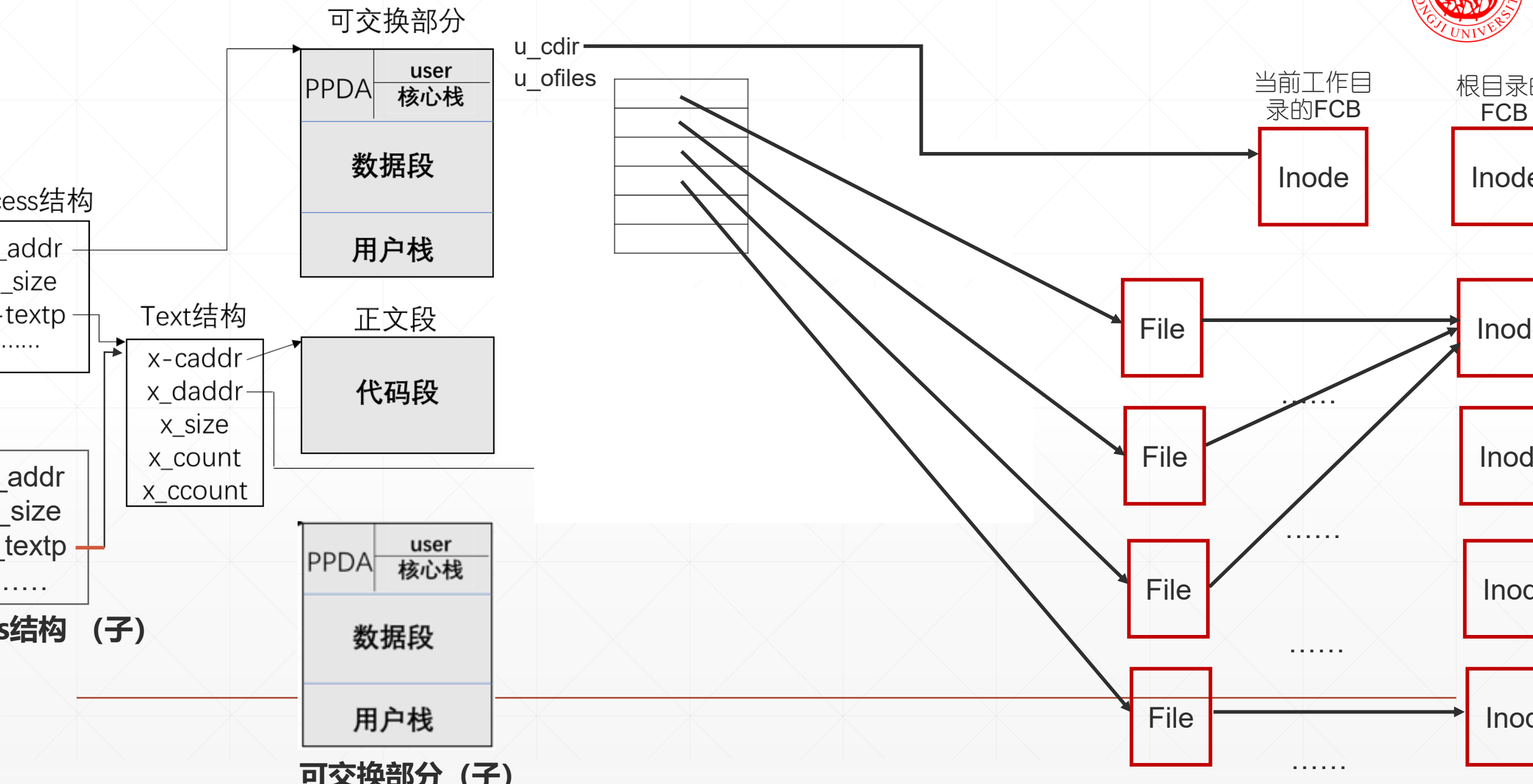
进程图像使用的系统资源

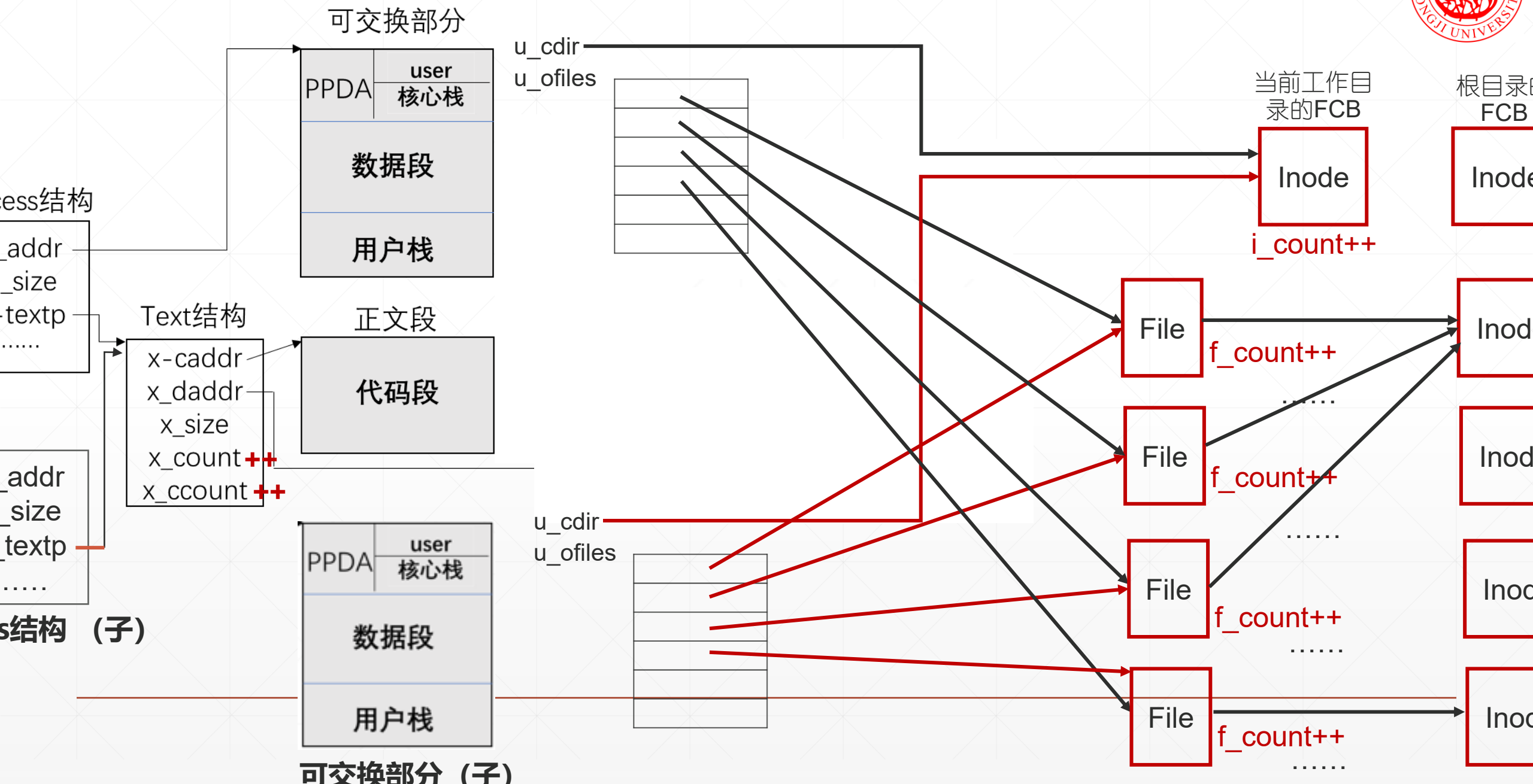


- 进程标识符 pid, 1个独用的整数
- 系统区
 - Process数组分配1个空元素
 - 页表区, 物理内存[2M,4M), 分配2张页表
- 用户区2个连续物理内存块
 - 共享正文段, x_size字节
 - 可交换部分, p_size字节









管理、调度内存和盘交换上的进程图像

为进程提供数据服务（数据读取）

交换部分

user
核心栈

数据段

户栈

文段

代码段

u_procp

u_MemoryDescriptor

u_cdir

u_ofiles

```
class MemoryDescriptor
```

```
{  
public:
```

```
PageTable*
```

```
m_UserPageTableArray; // 进程的页表
```

```
// ----- 虚空间布局 -----
```

```
unsigned long m_TextStartAddress; // 代码段起始地址（虚地址）
```

```
unsigned long m_TextSize; // 代码段长度
```

```
unsigned long m_DataStartAddress; // 数据段起始地址（虚地址）
```

```
unsigned long m_DataSize; // 数据段长度
```

```
unsigned long m_StackSize; // 栈段长度（虚地址）
```

```
}
```

当前工作目
录的FCB

Inode

根目录的
FCB

Inode

File

Inode

File

Inode

File

Inode

File

Inode

文件会话状态

系统正在

管理、调度内存和盘交换上的进程图像

为进程提供数据服务（数据读取）

交换部分

user
核心栈

数据段

户栈

文段

代码段

u_procp

u_MemoryDescriptor

u_cdir

u_ofiles

```
class MemoryDescriptor
```

```
{  
public:
```

```
PageTable*
```

```
m_UserPageTableArray; // 进程的页表
```

```
// ----- 虚空间布局 -----
```

```
unsigned long m_TextStartAddress; // 代码段起始地址（虚地址）
```

```
unsigned long m_TextSize; // 代码段长度
```

```
unsigned long m_DataStartAddress; // 数据段起始地址（虚地址）
```

```
unsigned long m_DataSize; // 数据段长度
```

```
unsigned long m_StackSize; // 栈段长度（虚地址）
```

```
}
```

当前工作目
录的FCB

Inode

根目录的
FCB

Inode

File

Inode

File

Inode

File

Inode

File

Inode

文件会话状态

系统正在